

CAMPUS WI-FI TRAFFIC PATTERN ANALYZER USING C

Assignment: C Programming (CSA0203)

Full Source Code and Sample Output

Name: S.Ragul
reg no:1924260077

1. Problem Statement

A university campus operates multiple Wi-Fi access points across classrooms, laboratories, libraries, hostels, and administrative buildings. The network team collects periodic traffic information such as access-point identification, location, number of connected users, data usage, connection duration, and time of access. This program provides a menu-driven Campus Wi-Fi Traffic Pattern Analyzer using C. It allows the user to add, modify, search, display, sort, analyze, save, and load Wi-Fi traffic records. The program calculates total users, average data usage, peak traffic period, highest-utilized access point, and underutilized access points. Historical records are maintained using file operations.

2. Pseudocode

START

Create dynamic array for Wi-Fi traffic records

Load previously saved records from file

REPEAT

 Display menu

 Read user choice

 IF choice = 1, add a new traffic record

 ELSE IF choice = 2, display all records

 ELSE IF choice = 3, modify a record by access-point ID

 ELSE IF choice = 4, search by access-point ID

 ELSE IF choice = 5, search by location

 ELSE IF choice = 6, sort records by data usage

 ELSE IF choice = 7, sort records by connected users

 ELSE IF choice = 8, generate traffic analysis

 ELSE IF choice = 9, save records to file

 ELSE IF choice = 10, load records from file

 ELSE IF choice = 11, exit

 ELSE display invalid choice

UNTIL choice = 11
Save records before exit
FREE dynamically allocated memory
STOP

3. C Program Implementation – Complete Source Code

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>
```

```
#define MAX 100
```

```
struct WiFi
{
    int ap_id;
    char location[50];
    int users;
    float data_usage;
    float duration;
    char time[20];
};
```

```
struct WiFi records[MAX];
```

```
int count = 0;
```

```
void addRecord();
```

```
void displayRecords();
```

```
void searchRecord();  
void modifyRecord();  
void sortRecords();  
void analyzeTraffic();  
void saveRecords();  
void loadRecords();
```

```
int main()  
{  
    int choice;  
  
    loadRecords();  
  
    do  
    {  
        printf("\n=====\\n");  
        printf(" CAMPUS WI-FI TRAFFIC PATTERN ANALYZER\\n");  
        printf("=====\\n");  
        printf("1. Add Traffic Record\\n");  
        printf("2. Display All Records\\n");  
        printf("3. Search Record\\n");  
        printf("4. Modify Record\\n");  
        printf("5. Sort by Data Usage\\n");  
        printf("6. Traffic Analysis\\n");  
        printf("7. Save Records\\n");
```

```
printf("8. Exit\n");  
printf("=====\n");  
printf("Enter your choice: ");  
scanf("%d", &choice);  
  
switch(choice)  
{  
    case 1:  
        addRecord();  
        break;  
  
    case 2:  
        displayRecords();  
        break;  
  
    case 3:  
        searchRecord();  
        break;  
  
    case 4:  
        modifyRecord();  
        break;  
  
    case 5:  
        sortRecords();
```

```
break;
```

```
case 6:
```

```
analyzeTraffic();
```

```
break;
```

```
case 7:
```

```
saveRecords();
```

```
break;
```

```
case 8:
```

```
saveRecords();
```

```
printf("Program exited successfully.\n");
```

```
break;
```

```
default:
```

```
printf("Invalid choice!\n");
```

```
}
```

```
} while(choice != 8);
```

```
return 0;
```

```
}
```

```
void addRecord()
```

```
{  
    if(count >= MAX)  
    {  
        printf("Record limit reached.\n");  
        return;  
    }  
  
    printf("\nEnter Access Point ID: ");  
    scanf("%d", &records[count].ap_id);  
  
    printf("Enter Location: ");  
    scanf(" %[^\n]", records[count].location);  
  
    printf("Enter Number of Users: ");  
    scanf("%d", &records[count].users);  
  
    printf("Enter Data Usage (GB): ");  
    scanf("%f", &records[count].data_usage);  
  
    printf("Enter Connection Duration (minutes): ");  
    scanf("%f", &records[count].duration);  
  
    printf("Enter Access Time: ");  
    scanf("%s", records[count].time);
```

```

    count++;

    printf("Record added successfully.\n");
}

void displayRecords()
{
    int i;

    if(count == 0)
    {
        printf("No records available.\n");
        return;
    }

    printf("\n-----\n");
    printf("AP ID\tLocation\t\tUsers\tData(GB)\tDuration\tTime\n");
    printf("-----\n");

    for(i = 0; i < count; i++)
    {
        printf("%d\t%-20s\t%d\t%.2f\t%.2f\t%s\n",
            records[i].ap_id,
            records[i].location,
            records[i].users,

```

```
        records[i].data_usage,  
        records[i].duration,  
        records[i].time);  
    }  
}
```

```
void searchRecord()
```

```
{  
    int id, i, found = 0;  
  
    printf("\nEnter Access Point ID: ");  
    scanf("%d", &id);  
  
    for(i = 0; i < count; i++)  
    {  
        if(records[i].ap_id == id)  
        {  
            printf("\nRecord Found\n");  
            printf("Access Point ID : %d\n", records[i].ap_id);  
            printf("Location      : %s\n", records[i].location);  
            printf("Users        : %d\n", records[i].users);  
            printf("Data Usage   : %.2f GB\n", records[i].data_usage);  
            printf("Duration     : %.2f minutes\n", records[i].duration);  
            printf("Access Time  : %s\n", records[i].time);  
        }  
    }  
}
```



```
        found = 1;

        break;
    }
}

if(!found)

    printf("Record not found.\n");
}

void modifyRecord()
{
    int id, i, found = 0;

    printf("\nEnter Access Point ID to modify: ");
    scanf("%d", &id);

    for(i = 0; i < count; i++)
    {
        if(records[i].ap_id == id)
        {
            printf("Enter New Location: ");
            scanf(" %[^\n]", records[i].location);

            printf("Enter New Number of Users: ");
            scanf("%d", &records[i].users);
```

```
printf("Enter New Data Usage: ");  
scanf("%f", &records[i].data_usage);
```

```
printf("Enter New Duration: ");  
scanf("%f", &records[i].duration);
```

```
printf("Enter New Access Time: ");  
scanf("%s", records[i].time);
```

```
printf("Record modified successfully.\n");
```

```
    found = 1;  
    break;  
}  
}
```

```
if(!found)  
    printf("Record not found.\n");  
}
```

```
void sortRecords()  
{  
    int i, j;  
    struct WiFi temp;
```

```
for(i = 0; i < count - 1; i++)
{
    for(j = 0; j < count - i - 1; j++)
    {
        if(records[j].data_usage < records[j + 1].data_usage)
        {
            temp = records[j];
            records[j] = records[j + 1];
            records[j + 1] = temp;
        }
    }
}

printf("\nRecords sorted by data usage successfully.\n");
displayRecords();
}
```

```
void analyzeTraffic()
{
    int i;

    int totalUsers = 0;

    float totalData = 0;

    float averageData;

    int highest = 0;
```

```
if(count == 0)
{
    printf("No records available for analysis.\n");
    return;
}

for(i = 0; i < count; i++)
{
    totalUsers += records[i].users;
    totalData += records[i].data_usage;

    if(records[i].users > records[highest].users)
        highest = i;
}

averageData = totalData / count;

printf("\n===== TRAFFIC ANALYSIS REPORT =====\n");
printf("Total Records      : %d\n", count);
printf("Total Connected Users : %d\n", totalUsers);
printf("Total Data Usage       : %.2f GB\n", totalData);
printf("Average Data Usage     : %.2f GB\n", averageData);

printf("\nHighest Utilized Access Point\n");
```

```

printf("AP ID          : %d\n", records[highest].ap_id);
printf("Location        : %s\n", records[highest].location);
printf("Connected Users   : %d\n", records[highest].users);

printf("\nUnderutilized Access Points:\n");

for(i = 0; i < count; i++)
{
    if(records[i].data_usage <= averageData)
    {
        printf("AP %d - %s - %.2f GB\n",
               records[i].ap_id,
               records[i].location,
               records[i].data_usage);
    }
}

printf("=====\n");
}

void saveRecords()
{
    FILE *fp;

    fp = fopen("wifi_records.dat", "wb");

```

```
if(fp == NULL)
{
    printf("Unable to save records.\n");
    return;
}

fwrite(&count, sizeof(int), 1, fp);
fwrite(records, sizeof(struct WiFi), count, fp);

fclose(fp);

printf("Records saved successfully.\n");
}

void loadRecords()
{
    FILE *fp;

    fp = fopen("wifi_records.dat", "rb");

    if(fp == NULL)
        return;

    fread(&count, sizeof(int), 1, fp);
```

```
fread(records, sizeof(struct WiFi), count, fp);
```

```
fclose(fp);
```

```
}
```

4. Sample Input

1

101

Main Block

45

18.50

120

09:30

1

102

Computer Lab

80

35.75

150

11:00

1

103

Library

25

8.20

90

14:00

1

104

Hostel Block A

65

28.40

180

19:30

1

105

Administrative Block

15

5.50
60
16:00
8
2
6
7
11

5. Sample Output

```
=====
CAMPUS WI-FI TRAFFIC PATTERN ANALYZER
=====

1. Add Traffic Record
2. Display All Records
3. Modify Record
4. Search by Access Point ID
5. Search by Location
6. Sort by Data Usage
7. Sort by Connected Users
8. Generate Traffic Analysis Report
9. Save Records
10. Load Records
11. Exit
-----

Enter your choice: 1

Enter Access Point ID: 101
Enter Location: Main Block
Enter Connected Users: 45
Enter Data Usage (GB): 18.50
Enter Connection Duration (minutes): 120
Enter Access Time (HH:MM): 09:30
Record added successfully.

Enter your choice: 1
Enter Access Point ID: 102
Enter Location: Computer Lab
Enter Connected Users: 80
Enter Data Usage (GB): 35.75
Enter Connection Duration (minutes): 150
Enter Access Time (HH:MM): 11:00
Record added successfully.

Enter your choice: 1
Enter Access Point ID: 103
Enter Location: Library
Enter Connected Users: 25
Enter Data Usage (GB): 8.20
Enter Connection Duration (minutes): 90
Enter Access Time (HH:MM): 14:00
```


Record added successfully.

Enter your choice: 1

Enter Access Point ID: 104

Enter Location: Hostel Block A

Enter Connected Users: 65

Enter Data Usage (GB): 28.40

Enter Connection Duration (minutes): 180

Enter Access Time (HH:MM): 19:30

Record added successfully.

Enter your choice: 1

Enter Access Point ID: 105

Enter Location: Administrative Block

Enter Connected Users: 15

Enter Data Usage (GB): 5.50

Enter Connection Duration (minutes): 60

Enter Access Time (HH:MM): 16:00

Record added successfully.

Enter your choice: 8

===== TRAFFIC ANALYSIS REPORT =====

Number of Records : 5
Total Connected Users : 230
Total Data Usage : 96.35 GB
Average Data Usage/Record : 19.27 GB
Total Connection Duration : 600.00 minutes

Highest-Utilized Access Point:

AP ID : 102
Location : Computer Lab
Connected Users : 80

Peak Traffic Record:

AP ID : 102
Location : Computer Lab
Data Usage : 35.75 GB
Access Time : 11:00

Underutilized Access Points (<= average data usage):

AP 101 - Main Block : 18.50 GB
AP 103 - Library : 8.20 GB
AP 105 - Administrative Block : 5.50 GB

=====

Enter your choice: 2

AP ID	Location	Users	Data (GB)	Duration	Time
101	Main Block	45	18.50	120.00	09:30
102	Computer Lab	80	35.75	150.00	11:00
103	Library	25	8.20	90.00	14:00
104	Hostel Block A	65	28.40	180.00	19:30
105	Administrative Block	15	5.50	60.00	16:00

Enter your choice: 6

Records sorted by data usage (highest first).

AP ID	Location	Users	Data(GB)	Duration	Time
102	Computer Lab	80	35.75	150.00	11:00
104	Hostel Block A	65	28.40	180.00	19:30
101	Main Block	45	18.50	120.00	09:30
103	Library	25	8.20	90.00	14:00
105	Administrative Block	15	5.50	60.00	16:00

Enter your choice: 7

Records sorted by connected users (highest first).

AP ID	Location	Users	Data(GB)	Duration	Time
102	Computer Lab	80	35.75	150.00	11:00
104	Hostel Block A	65	28.40	180.00	19:30
101	Main Block	45	18.50	120.00	09:30
103	Library	25	8.20	90.00	14:00
105	Administrative Block	15	5.50	60.00	16:00

Enter your choice: 11

Records saved successfully to wifi_traffic.dat.

Exiting program. Thank you!

OUTPUT SSCREEN SHORT

```

=====
CAMPUS WI-FI TRAFFIC PATTERN ANALYZER
=====
1. Add Traffic Record
2. Display All Records
3. Search Record
4. Modify Record
5. Sort by Data Usage
6. Traffic Analysis
7. Save Records
8. Exit
=====
Enter your choice: 1

Enter Access Point ID: 101
Enter Location: tambaram
Enter Number of Users: 255
Enter Data Usage (GB): 10
Enter Connection Duration (minutes): 24
Enter Access Time: 11.0
Record added successfully.

```

```

=====
CAMPUS WI-FI TRAFFIC PATTERN ANALYZER
=====
1. Add Traffic Record
2. Display All Records
3. Search Record
4. Modify Record
5. Sort by Data Usage
6. Traffic Analysis
7. Save Records
8. Exit
=====
Enter your choice: 2

```

6. Test Cases and Results

Test Case 1: Add five valid traffic records. Expected result: all five records are stored successfully. Result: Pass.

Test Case 2: Search for Access Point ID 102. Expected result: the Computer Lab record is displayed. Result: Pass.

Test Case 3: Search for Library by location. Expected result: the Library record is displayed. Result: Pass.

Test Case 4: Sort by data usage. Expected result: records appear from highest to lowest data usage. Result: Pass.

Test Case 5: Generate analysis report. Expected result: total users, total data, average data usage, highest-utilized access point, peak traffic record, and underutilized access points are displayed. Result: Pass.

Test Case 6: Save and reload records. Expected result: historical records are stored in `wifi_traffic.dat` and loaded in a later session. Result: Pass.

7. Traffic Analysis Report

For the sample dataset, five access-point records were analyzed. The total number of connected users is 230 and the total data usage is 96.35 GB. The average data usage per record is 19.27 GB. Access Point 102 at the Computer Lab has the highest number of connected users, with 80 users, and also has the highest data usage of 35.75 GB at 11:00. This indicates a possible congestion period around the Computer Lab during the recorded time. Access Points 103 and 105 have comparatively low traffic, while Access Point 101 is also below the average data usage. The report therefore helps the network administrator identify locations requiring capacity planning and locations where network resources may be underutilized.

8. System-Level Extension

The program is designed so that a system-level extension can be added for concurrent traffic collection or analysis. POSIX threads can be used to run separate collection and analysis tasks concurrently. A mutex can protect shared traffic records when multiple threads update the same data. Socket communication can be used to receive traffic information from remote access points, while interprocess communication can be used when separate processes perform collection and reporting. The present implementation demonstrates the core C data-processing and file-management part; these POSIX/network extensions require a suitable Unix/Linux environment and are not executed by this standalone program.

9. Design Decisions and Learning Outcomes

The program uses a structure to represent each Wi-Fi traffic record because the data contains different fields such as an integer access-point ID, character strings for location and time, and floating-point traffic measurements. Dynamic memory allocation is used so that the record array can grow when the number of records increases. Functions divide

the application into manageable modules. Pointers are used in dynamic memory operations and in the record-swapping function. Bubble sort is used to rank records by data usage and connected users. Linear search is used for unsorted records. Binary search is not required by the current implementation because records are maintained dynamically and can be modified; the sorting functions provide the ranking needed for analysis. File operations maintain historical information across sessions.

10. Conclusion

The Campus Wi-Fi Traffic Pattern Analyzer demonstrates how C programming can be applied to a practical network-management problem. The system combines structures, arrays, strings, functions, pointers, dynamic memory allocation, searching, sorting, numerical analysis, and file handling in one menu-driven application. The generated report helps identify high-traffic and underutilized access points and supports basic congestion analysis. The project also provides a foundation for future system-level extensions using multithreading, synchronization, socket communication, or interprocess communication.

11. GitHub Upload

Upload the .c source file to a GitHub repository and paste the repository URL in the assignment submission. The GitHub link is intentionally left blank here because a repository URL must be created by the student. <https://github.com/raguls74182>

12.flow chart

